

CPSC 250

Multiple Inheritance and Mixin Classes

Edward Brash

1 Multiple Inheritance

Python allows a class to inherit from more than one base class.

This is called **multiple inheritance**.

For example:

```
class Bat(Winged, Mammal):  
    pass
```

A bat is both winged and a mammal, so it may make sense for it to inherit from both classes.

2 Conceptual Example

Suppose:

```
class Winged:  
  
    def flap_wings(self):  
        print("flap flap")  
  
class Mammal:  
  
    def breathe(self):  
        print("breathing")  
  
    def give_birth(self):  
        print("giving birth")
```

Then:

```
class Bat(Winged, Mammal):  
    pass
```

A Bat object can use methods inherited from both parent classes.

3 Method Name Conflicts

A complication arises if both base classes define a method with the same name.

For example, suppose both parent classes define:

```
blink()
```

Which version does Python use?

4 Method Resolution Order

Python uses the method from the first base class listed, according to its method resolution order.
For example:

```
class Bat(Winged, Mammal):  
    pass
```

If both `Winged` and `Mammal` define `blink()`, Python will search `Winged` first.

5 Mixin Classes

A mixin is a class designed to add a small piece of behavior to another class.
Mixin classes are not usually intended to stand alone.
They are used to compose behavior.

6 Transport Example

Suppose we begin with a base class:

`TransportMode`

Then we want to create:

- a `Truck`,
- a `FlyingCar`.

Both are transport modes, but they move differently.

7 Driving Mixin

```
class DrivingMixin:  
  
    def drive(self, distance):  
        print(f"Driving {distance} miles")
```

8 Flying Mixin

```
class FlyingMixin:  
  
    def fly(self, distance, altitude):  
        print(f"Flying {distance} miles at {altitude} feet")
```

9 Using Mixins

A truck can drive:

```
class Truck(TransportMode, DrivingMixin):  
  
    def go(self, distance):  
        self.drive(distance)
```

A flying car can both fly and drive:

```
class FlyingCar(TransportMode, DrivingMixin, FlyingMixin):  
  
    def go(self, distance):  
        self.fly(distance / 2, self.altitude)  
        self.drive(distance / 2)
```

10 Why Mixins Are Useful

Mixins allow us to keep behavior modular.

Instead of copying the same `drive()` method into several classes, we define it once and inherit it where needed.

This supports:

- code reuse,
- cleaner design,
- better encapsulation,
- simpler maintenance.

11 Using the Objects

The end user should be able to write:

```
s = Truck(...)  
f = FlyingCar(...)  
  
s.go(100)  
f.go(100)
```

and have both objects behave appropriately.

This is another example of polymorphism.

12 Key Takeaways

- Python supports multiple inheritance.
- A class can inherit from more than one parent class.
- If method names conflict, Python follows method resolution order.
- Mixins add focused behavior to classes.

- Mixins help avoid duplicate code.
- Polymorphism allows different objects to use the same interface.